

Structured Programming

- [Structured Programming](#)
 - [Introduction](#)
 - [Learning Objectives](#)
 - [Task 1 Setup Branches](#)
 - [Task 2 LED bar](#)
 - [Task 3 Sizedisplay](#)
 - [Task 4 Count zeroes](#)

Introduction

In this lab you will use structured programming techniques to implement an assembly program. Depending on the state of button T0, the potentiometer POT1 and the DIP switches **S7..S0** are read and compared. The results are displayed in various ways on the 7-segment display and the **LED31..0**.

Learning Objectives

- You are able to structure a given problem using a diagram.
- You are able to convert from a diagram into readable assembly code.

Task 1 Setup Branches

Draw a diagram with the following functionality and then implement it in assembly. * The program shall read the ADC-value using the given function **adc_get_value()** and store the value in a register. * The program shall check the state of button T0. * **If T0 is pressed** the background color of the LCD shall be set to green and the read ADC-value shall be displayed on the 7-segment display. * **If T0 is *not* pressed** the program shall read an 8-bit value from DIP switches **S7..S0**. The ADC value shall be subtracted from the DIP switch value. The resulting difference shall be displayed on the 7-segment display. * **If diff ≥ 0** the background color of the LCD shall be set to blue. * **If diff < 0** the background color of the LCD shall be set to red.

Verify that all cases are implemented correctly.

Here are some options for software to draw diagrams:

- [Umlet](#) - for all kinds of diagrams (old but good)
- [structorizer](#) - for structograms...
- [graphviz](#) - for version controllable diagrams (steep learning curve)
- [mermaid](#) - another plain text to diagram option (web application with AI stuff.. people seem to like it)

- ADC stands for Analog/Digital Converter. An ADC converts an analog value into a binary representation. In our case the voltage on the potentiometer is read (voltage divider). The voltage is represented as 8-bit value. A voltage of 0 Volt corresponds to an ADC-value of 0, whereas a voltage of 3.3 Volt corresponds to an ADC-value of 255.
- The function call `bl adc_get_value` will return the ADC-value in the 32-bit register `r0`.
- There is a special register address available to output the content of a register as HEX on the 7-segment display. See ‘7-Segment Binary Interface’ on the [CT-Wiki][link-ennis].

Task 2 LED bar

Extend the green branch (branch which sets the LCD backlight to green) of your diagram and assembly program (in that order). Display a LED bar on `LEDS 31..0`, whose length scales with the ADC value.

```
1 bar_length = (adc_value / 8) + 1
```

Remember, that a right shift is a division by $2^{(\text{shift width})}$.

Task 3 Sizedisplay

Extend the blue branch (branch which sets the LCD backlight to blue) of your diagram and assembly program (in that order). The program shall evaluate the difference calculated before. Display the number of bits required to represent the difference on the LCD:

- 2 bit, i.e. `diff < 4`, display ‘2 Bit’ on the LCD
- 4 bit, i.e. `diff < 16`, display ‘4 Bit’ on the LCD
- 8 bit, i.e. otherwise, display ‘8 Bit’ on the LCD

Use the given function `write_bit_ascii` to output the string ‘Bit’ to the LCD. Then write the determined size to the LCD.

Task 4 Count zeroes

Extend the red branch (branch which sets the LCD backlight to red) of your diagram and assembly program (in that order). The program shall count the number of zeroes in the binary representation of the difference calculated before. The number of zeroes shall be displayed on the second line of the LCD.

Grading

Criteria	Weight
The diagram and program meet the requirements of Task 1 .	1/4
Program meets the functionality added in Task 2 .	1/4
Program meets the functionality added in Task 3 .	1/4
Program meets the functionality added in Task 4 .	1/4